```
address after change:  140509666477824

# pop an element from the between of the array
print("address before change: ", id(list1))
list1.insert(3, 30)
print(list1)
print("address after change: ", id(list1))
Output
address before change:  140509666477824
[100, 2, 8, 30, 9, 10, 11, 12]
address after change:  140509666477824
```

Sort

The address of the list doesn't get changed before and after the sort operation.

```
# sort the array
print("address before change: ", id(list1))
list1.sort()
print(list1)
print("address after change: ", id(list1))
```

The output is given below.

Note: Even after sort, the address remains the same.

```
address before change:  140509666477824
[2, 8, 9, 10, 11, 12, 30, 100]
address after change:  140509666477824
```

Performance

- Append - O(1)
- Extend - O(k)
- Pop last element - O(1)
- Pop anywhere else - O(n) worst case
- Insert - O(n) worst case
- Index - O(1)
- Slice - O(k)
- in Operator - O(n)
- Sort - O(nlogn) - tim sort is used

Here, n = number of elements; k = k'th index; 1 = order of 1.

Tuples

Let's observe how tuples are defined, and how they differ in the allocation of memory compared to lists. Tuples are:

- Immutable in nature
- Consume less memory as compared to lists

Definition

Here's a quick example of how a tuple is defined:

```
# example of tuple
tuple1 = ('one', 'two', 'three')
print(type(tuple1))
print(tuple1)
```

The output is:

```
<class 'tuple'>
('one', 'two', 'three')
```

Slicing

Accessing tuples is the same as lists:

```
# access elements using index ; first 2 indexes
print(tuple1[0:2])

# access the last index of the tuple
print(tuple1[-1])
```

The output is:

```
('one', 'two')
three
```

Changing the single value

As tuples are immutable in nature, we cannot change their value. You can find the error that comes up while trying to change the value of the tuple as follows:

```
# tuple are immutable
tuple1[0] = "cherry"
print(type(tuple1))
print(tuple1)
```

The output is:

```
TypeError                                 Traceback (most
recent call last)
<ipython-input-13-a780f574006b> in <module>
      1 # tuple are immutable
----> 2 tuple1[0] = "cherry"
      3 print(type(tuple1))
      4 print(tuple1)

TypeError: 'tuple' object does not support item assignment
```

Replacing a tuple with a new tuple

We can overwrite the existing tuple to get a new tuple; the address will also be overwritten:

```
# but we can assign the tuple1 variable to any other value
also another tuple
```